

USE CALLBACK() & USE MEMO() - COMPLETE NOTES Page 1

1. What is useCallback() ?

- `useCallback()` returns a memoized (cached) version of a function.
- The function reference stays the same until the dependencies change.
- Prevents unnecessary re-creations of functions.

`useCallback`
= Cache
Function
Reference
☆

2. What is useMemo() ?

- `useMemo()` returns a memoized (cached) value.
- Re-computes the value only when dependencies change.
- Prevents expensive calculations on every render.

`useMemo`
= Cache
Computed
Value
☆

3. When to Use ?

Hook	Use When...
<code>useCallback()</code>	You want to cache/keep the same function reference between renders.
<code>useMemo()</code>	You want to cache expensive calculations and avoid re-computing.

4. Syntax

`useCallback()`

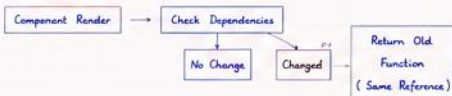
```
const memoizedFunction = useCallback(  
  () => {  
    // function logic  
  }, [dependencies]  
);
```

`useMemo()`

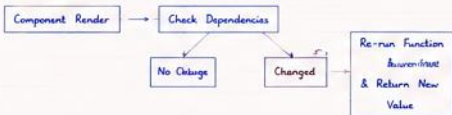
```
const memoizedValue = useMemo(  
  () => {  
    // expensive calculation  
    return result;  
  }, [dependencies]  
);
```

5. How it Works ?

`useCallback()`



`useMemo()`



Both hooks
use dependency
array to decide
when to update
or reuse.



6. Why Do We Need These Hooks ?

- ✓ Prevent unnecessary re-renders in child components.
- ✓ Improve performance.
- ✓ Avoid recreating functions / recalculating values on each render.
- ✓ Useful in large or complex applications.

Goal:
Optimize
Renders &
Improve
Performance
☆

7. Example - useCallback()

```
// Without useCallback()
function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = () => { // new function on every render
    console.log('Clicked', count);
  };

  return <Child handleClick={handleClick} />;
}

// With useCallback()
function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => { // same function until count changes
    console.log('Clicked', count);
  }, [count]);

  return <Child handleClick={handleClick} />;
}
```

Helps when
passing functions
to child
components
or using in
useEffect().



8. Example - useMemo()

```
// Without useMemo()
const total = items.reduce((acc, item) => acc + item.price, 0); // runs on every render

// With useMemo()
const total = useMemo(() => {
  return items.reduce((acc, item) => acc + item.price, 0);
}, [items]); // runs only when items changes
```

Great for
expensive
calculations



9. Dependency Array Rules

- [] → run once (only on mount)
- [value] → run when that value changes
- [value1, value2] → run when any of them changes
- No dependency array → runs on every render (Not recommended)

10. Important Points

- ✓ useCallback() returns a function reference.
- ✓ useMemo() returns a computed value.
- ✓ Don't use them everywhere. Use only when needed.
- ✓ Overuse can make code complex.

Use Smartly,
Not
Everywhere



11. Common Use Cases

<u>useCallback()</u>	<u>useMemo()</u>
• Passing functions to child components (with <u>React.memo()</u>) • Functions used in <u>useEffect()</u> dependency array • Event handlers in optimized components	• Expensive calculations • Filtering, sorting large lists • Derived data from props/state • Prevent re-computation

Mini Summary

useCallback() → Memoize Function (Reference)

useMemo() → Memoize Value (Result)

Both help in performance optimization & prevent unnecessary renders.



12. useCallback() vs useMemo()

Feature	<u>useCallback()</u>	<u>useMemo()</u>
Returns	Memoized Function	Memoized Value
Purpose	Keep same function reference	Cache computed value
When to Use	When passing functions to child or <code>useEffect()</code>	When doing expensive calculations
Dependency	Array of dependencies	Array of dependencies
Re-render Prevention	Yes (Prevents child re-render)	Yes (Prevents re-calculation)
Example	<code>useCallback(() => {...}, [deps])</code>	<code>useMemo(() => {...}, [deps])</code>

13. Best Practices

- ✓ Use dependency array correctly.
- ✓ Avoid empty dependency [] unless needed.
- ✓ Don't put objects/arrays directly in dependency (`useMemo/useCallback` them).
- ✓ Keep functions and calculations outside component if possible.
- ✓ Profile performance before & after using.

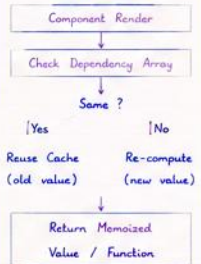
Correct usage
= Better
Performance
& Clean Code



14. Quick Revision

1. `useCallback()` → Cache function reference.
2. `useMemo()` → Cache computed value.
3. Both use dependency array.
4. Both help in performance optimization.
5. Use only when necessary.
6. Goal → Prevent unnecessary renders & calculations ☆

Complete Flow



15. Practical Example - Both Together

```

function Parent({ list }) {
  const [query, setQuery] = useState("");

  const filteredList = useMemo(() => {
    list.filter(item => item.name.toLowerCase().includes(query.toLowerCase()));
  }, [list, query]);

  const handleSearch = useCallback((e) => {
    setQuery(e.target.value);
  }, []); // stable function reference

  return (
    <div>
      <input value={query} onChange={handleSearch} placeholder="Search..." />
      <Child items={filteredList} />
    </div>
  );
}
  
```

Fitter only
when list
or query
changes.
Handler ref
stays same.
Performance
Improved



★ Remember → Optimize, Don't Over-Optimize ★